Project 2 — NLP / LLM System

Ý tưởng: Xây dựng một hệ thống xử lý và hiểu **text tiếng Việt** để giải quyết một bài toán NLP thực tế, thay vì chỉ làm chatbot.

Ví dụ: **Vietnamese Document Intelligence System**.

Người dùng đưa vào các tài liệu như:

- tài liệu kỹ thuật
- quy định
- hợp đồng
- giáo trình
- báo cáo
- tài liệu doanh nghiệp

Sau đó hệ thống có thể:

- tìm kiếm thông tin liên quan
- trả lời câu hỏi
- tóm tắt tài liệu
- trích xuất thông tin
- phát hiện quan hệ giữa các thông tin
- cung cấp evidence cho câu trả lời

Project có thể sử dụng **RAG + embedding + vector database + reranking + LLM**, nhưng trọng tâm không phải "tôi làm RAG", mà là:

> **Làm thế nào để hệ thống hiểu và truy xuất thông tin từ tài liệu tiếng Việt một cách chính xác?**

Bạn có thể tự xây dataset/evaluation set, thử nhiều retrieval strategies, embedding models, rerankers, prompting/fine-tuning và phân tích lỗi.

Điểm nhấn: NLP + ML/LLM + Information Retrieval + RAG + Evaluation.

Phase 0 — Định nghĩa project

Bước 1 — Chọn một bài toán cụ thể

Đừng bắt đầu bằng:

> "Tôi muốn làm RAG."

Hãy bắt đầu bằng:

> "Tôi muốn giải quyết vấn đề gì?"

Ví dụ:

Vietnamese Technical Document Intelligence

Người dùng upload:

- PDF
- DOCX
- text documents

Sau đó có thể hỏi:

> "Component X có chức năng gì?"
>
> "Quy trình xử lý lỗi Y là gì?"
>
> "Điều kiện để thực hiện Z là gì?"

Hệ thống trả lời:

Answer
+
Evidence
+
Source document
+
Page / section

Phase 1 — Research bài toán

Bước 2 — Tìm hiểu domain

Trước khi code, bạn phải hiểu dữ liệu mình xử lý.

Ví dụ chọn:

> Vietnamese technical documents

Bạn cần tìm hiểu:

- tài liệu có cấu trúc như thế nào?
- heading?
- paragraph?
- table?
- bullet?
- PDF scan?
- OCR?
- tiếng Việt có vấn đề gì?
- tài liệu dài bao nhiêu?
- query thường có dạng gì?

Mục đích:

> **Hiểu problem trước khi chọn technology.**

Phase 2 — Literature / Technology Research

Bước 3 — Tìm hiểu những phương pháp hiện có

Đây là lúc bạn bắt đầu học.

Bạn sẽ gặp những thứ như:

NLP

- tokenization
- TF-IDF
- Bag of Words
- embeddings
- semantic similarity

Information Retrieval

- inverted index
- BM25
- dense retrieval

- hybrid retrieval
- reranking

Deep Learning

- Transformer
- attention
- encoder/decoder

LLM

- prompting
- context window
- function calling
- structured output

RAG

- indexing
- retrieval
- generation

Chưa biết cái nào → học cái đó.

Không cần học toàn bộ NLP trước.

Phase 3 — Xây dataset

Đây là một trong những bước **quan trọng nhất**.

Những dataset tôi tìm được phù hợp nhất

Dataset	Data có gì	Dùng cho phần nào
Vietnamese Legal Retrieval	101,503 documents + 4,847 training examples + 539 test queries + relevance judgments	**Retrieval / embedding / reranker**
Vietnamese Legal QA	5,330 train QA + 592 test QA + legal corpus 207 MB	**Retrieval + RAG + QA evaluation**

UIT-ViQuAD 2.0	28,454 train + 3,814 validation + 7,301 test	**Question answering / reading comprehension**
Vietnamese Legal QA	9,715 legal documents + 29,145 generated QA	**Domain QA / RAG training**
Vietnamese Legal Documents	Corpus tài liệu pháp luật tiếng Việt	**Corpus để xây hệ thống retrieval riêng**
Vietnamese news classification	~1.3M bài báo, 11 topic	Classification phụ, **không phải dataset chính của Project 2**

Bước 4 — Thu thập documents

Bạn cần có corpus.

Ví dụ:

```
data/
├── manuals/
├── regulations/
├── technical_docs/
└── reports/
```

Có thể dùng:

- public documents
- government documents
- open datasets
- tài liệu public domain
- tự tạo dữ liệu nếu cần

Đừng lấy dữ liệu có vấn đề bản quyền để public lên GitHub.

Bước 5 — Làm data preprocessing

Bạn cần xây pipeline:

Raw PDF
↓
Text extraction
↓
Cleaning
↓
Normalization
↓
Structure extraction
↓
Metadata

Ví dụ metadata:

```
{
 "document_id": "doc_001",
 "page": 12,
 "section": "Maintenance",
 "text": "..."
}
```

Ở bước này bạn có thể gặp:

- PDF parsing
- OCR
- Unicode normalization
- Vietnamese text processing

Thiếu kiến thức nào → học.

Phase 4 — Xây Evaluation Dataset

Đây là bước mà rất nhiều project sinh viên bỏ qua.

Bước 6 — Tạo Question–Answer dataset

Bạn cần tạo những câu hỏi mà hệ thống phải trả lời.

Ví dụ:

Document:
technical_manual.pdf

Question:
"Valve X có chức năng gì?"

Answer:
"..."

Evidence:
Page 17, Section 3.2

Tạo khoảng một tập evaluation đủ đại diện cho các loại query bạn quan tâm.

Phân loại:

Fact retrieval
Definition
Multi-hop question
Comparison
Summarization
Numerical information
...

Đây sẽ là ground truth để benchmark hệ thống.

Phase 5 — Baseline 1: Keyword Search

Bước 7 — Xây search engine đơn giản nhất

Chưa dùng LLM.

User:

"Valve X có chức năng gì?"

System:

Query
↓
Keyword Search
↓
Relevant documents

Bạn có thể bắt đầu bằng:

TF-IDF

Sau đó:

BM25

Mục tiêu của bước này là có **baseline**.

Bạn cần đo:

- Precision
- Recall
- Precision@k
- Recall@k
- MRR

Nếu chưa biết các metric này → đây chính là lúc học Information Retrieval.

Phase 6 — Baseline 2: Dense Retrieval

Bước 8 — Dùng embedding

Pipeline:

Document
↓
Embedding Model
↓
Vector
↓
Vector Database / Index

Query:

Question
↓
Embedding
↓
Similarity Search
↓
Top-k documents

Bạn có thể bắt đầu với một embedding model pretrained.

Không cần train model.

Bước 9 — So sánh BM25 vs Dense Retrieval

Đây là experiment đầu tiên.

Ví dụ:

Method	Recall@5	MRR
TF-IDF	?	?
BM25	?	?
Dense	?	?

Đây là lúc project bắt đầu có **nghiên cứu/experiment**, chứ không chỉ là app.

Phase 7 — Chunking

Bước 10 — Nghiên cứu document chunking

Bạn sẽ nhận ra:

> Không thể embedding toàn bộ document dài.

Cần chunk.

Thử:

- fixed-size chunk
- sentence-based chunk
- paragraph-based chunk
- section-aware chunk

Ví dụ:

256 tokens
512 tokens
1024 tokens

Sau đó benchmark.

Bạn có thể phát hiện:

> 512 tốt hơn 256 trên dataset này.

Đó là **một finding của project**.

Phase 8 — Vector Database

Bước 11 — Xây document indexing pipeline

Bây giờ pipeline thành:

Documents
↓
Parse
↓
Clean
↓
Chunk
↓
Embedding
↓
Vector Index

Khi document mới được upload:

Upload
↓
Processing
↓
Chunking
↓
Embedding
↓
Index

Phase 9 — RAG

Bước 12 — Thêm LLM

Bây giờ mới bắt đầu RAG.

Pipeline:

User Question

↓
Retrieval
↓
Relevant chunks
↓
Context
↓
LLM
↓
Answer

Prompt yêu cầu:

- chỉ sử dụng context
- không tự bịa
- trả evidence
- nếu không có thông tin → nói không biết

Phase 10 — Đánh giá RAG

Bước 13 — So sánh

Bạn cần ít nhất:

LLM only
vs
LLM + RAG

Ví dụ:

System	Accuracy	Faithfulness
LLM only	?	?
RAG	?	?

Mục tiêu:

> Chứng minh RAG thực sự giúp bài toán của bạn.

Phase 11 — Hybrid Retrieval

Bước 14 — Kết hợp BM25 + Dense Retrieval

Bạn sẽ nhận ra:

BM25 tốt với:

- exact keyword

Dense retrieval tốt với:

- semantic meaning

Vậy thử:

BM25
+
Dense Retrieval
↓
Hybrid Retrieval

Benchmark:

BM25
Dense
Hybrid

Phase 12 — Reranking

Bước 15 — Thêm reranker

Pipeline:

Query
↓
Retriever
↓
Top 20
↓
Reranker
↓
Top 5
↓

LLM

So sánh:

Retriever
vs
Retriever + Reranker

Đo:

- Recall@k
- MRR
- answer accuracy

Phase 13 — Query Processing

Bước 16 — Làm query understanding

Người dùng có thể hỏi:

> "Còn cái này thì sao?"
>
> "Nó có chức năng gì?"
>
> "Điều kiện ở trên có áp dụng cho trường hợp này không?"

Query không đủ context.

Bạn có thể thêm:

Conversation History
↓
Query Rewriting
↓
Retrieval

LLM biến:

> "Còn cái này thì sao?"

thành:

> "Điều kiện bảo trì Valve X trong trường hợp Y là gì?"

Phase 14 — Advanced RAG

Bước 17 — Xem RAG baseline còn lỗi gì

Đây là lúc **error analysis**.

Lấy những câu hệ thống trả lời sai.

Phân loại:

Retrieval failure
Chunking failure
Query failure
Ranking failure
LLM reasoning failure
Hallucination

Đây là một bước cực kỳ quan trọng.

Phase 15 — Cải thiện hệ thống dựa trên lỗi

Bước 18 — Giải quyết từng failure mode

Ví dụ:

Retrieval failure

→ hybrid search

Wrong ranking

→ reranker

Context quá dài

→ better chunking

Query ambiguous

→ query rewriting

Hallucination

→ grounding + citation

Multi-hop question

→ multi-step retrieval

Phase 16 — Structured Output

Bước 19 — Không chỉ trả text

Thay vì:

Valve X được sử dụng để...

có thể trả:

```
{
 "answer": "...",
 "confidence": 0.87,
 "sources": [
   {
     "document": "manual.pdf",
     "page": 17
   }
 ]
}
```

Lúc này backend có thể sử dụng output của LLM một cách đáng tin cậy hơn.

Phase 17 — API

Bước 20 — Tách AI system khỏi UI

Architecture:

Frontend
 ↓
REST API

↓
AI Backend
↓
RAG Pipeline
↓
Models / Vector DB

Endpoints có thể gồm:

POST /documents
GET /documents
POST /search
POST /query
GET /documents/{id}

Phase 18 — Web Application

Bước 21 — Xây UI

User có thể:

1. Upload document
2. Xem document
3. Search
4. Ask question
5. Xem answer
6. Xem citation/evidence
7. Xem lịch sử query

Đây mới là lúc project trở thành một sản phẩm hoàn chỉnh.

Phase 19 — Testing

Bước 22 — Test từng tầng

Unit tests

Test:

- parser
- chunker

- retriever
- query processor

Integration tests

Upload
→ index
→ query
→ retrieval
→ answer

AI evaluation

Dùng evaluation dataset bạn đã tạo ở Phase 4.

Phase 20 — Experiment Tracking

Bước 23 — Theo dõi experiments

Bạn sẽ bắt đầu có rất nhiều experiment:

Embedding A
Embedding B

Chunk 256
Chunk 512

BM25
Dense
Hybrid

Reranker A
Reranker B

Đừng ghi bằng tay.

Dùng experiment tracking.

Lưu:

experiment
model
parameters
dataset version
metrics

result

Đây là lúc bạn có thể học **MLflow**.

Phase 21 — Data Versioning

Bước 24 — Version dataset

Ví dụ:

dataset v1
dataset v2
dataset v3

Khi dataset thay đổi, bạn phải biết:

> Model/evaluation result này được tạo từ dataset nào?

Đây là lúc có thể học:

DVC

Phase 22 — Containerization

Bước 25 — Dockerize

Có thể tách:

frontend
backend
vector-db
database

thành containers.

Ví dụ:

Docker Compose

frontend
backend

postgres
redis
vector database

Phase 23 — CI

Bước 26 — GitHub Actions

Mỗi khi push code:

Git push
↓
CI
↓
Lint
↓
Unit tests
↓
Integration tests
↓
Build

Nếu fail → không merge.

Phase 24 — CD

Bước 27 — Deployment

Pipeline:

GitHub
↓
GitHub Actions
↓
Docker Build
↓
Container Registry
↓
Cloud
↓
Deploy

Bạn có thể chọn cloud phù hợp với ngân sách.

Phase 25 — Monitoring

Bước 28 — Monitoring

Không chỉ monitor server.

AI system cần monitor:

System

- CPU
- RAM
- GPU
- latency
- errors

AI

- retrieval latency
- LLM latency
- token usage
- retrieval quality
- failed queries

Business/application

- number of queries
- successful answers
- unanswered questions

Phase 26 — AI-specific Observability

Bước 29 — Trace một request

Một query:

User query
↓

Query rewriting 120ms
↓
Embedding 80ms
↓
Vector search 30ms
↓
Reranking 150ms
↓
LLM 900ms
↓
Total 1.28s

Bạn biết chính xác bottleneck nằm ở đâu.

Phase 27 — Security

Bước 30 — Xử lý security

Đặc biệt nếu user upload document:

- authentication
- authorization
- file validation
- file size limits
- malicious files
- API rate limiting
- secrets management
- prompt injection
- data isolation

Nếu user A upload document:

user B không được retrieve nó.

Đây là một vấn đề rất thực tế của RAG.

Phase 28 — Final Evaluation

Bước 31 — Benchmark toàn bộ hệ thống

Cuối cùng bạn phải có bảng kiểu:

System	Recall@5	MRR	Answer Accuracy	Faithfulness	Latency
BM25					
Dense					
Hybrid					
Hybrid + Reranker					
Full RAG					

Sau đó:

Ablation study

Full System
↓
- Reranker
↓
- Hybrid Retrieval
↓
- Query Rewriting
↓
- RAG

Bạn biết thành phần nào thực sự có ích.

Phase 29 — Error Analysis

Bước 32 — Phân tích những câu sai

Không được chỉ báo:

Accuracy = 87%.

Bạn phải biết:

13% còn lại sai vì cái gì?

Ví dụ:

30% retrieval failure

25% ambiguous query
20% insufficient context
15% LLM reasoning
10% OCR

Đây là phần cực kỳ giá trị khi phỏng vấn.

Phase 30 — Documentation

Bước 33 — Viết README

README nên có:

1. Problem
2. Motivation
3. System overview
4. Architecture
5. Dataset
6. Models
7. Retrieval
8. RAG pipeline
9. Experiments
10. Evaluation
11. Error analysis
12. Deployment
13. Limitations
14. Future work

Phase 31 — Technical Report

Bước 34 — Viết report như một mini research project

Structure:

Abstract
Introduction
Problem Definition
Related Work
Methodology
System Architecture
Experiments

Results
Error Analysis
Limitations
Conclusion

Không cần publish paper.

Nhưng **cách tư duy nên gần với research**.

Phase 32 — Demo

Bước 35 — Làm demo

Một người vào web:

Upload document
↓
Ask question
↓
Answer
↓
Evidence

Bạn có thể quay một video demo ngắn.

Phase 33 — Chuẩn bị phỏng vấn

Bước 36 — Tự chất vấn project

Bạn phải trả lời được:

Problem

> Why did you choose this problem?

Data

> Where did your data come from?

Retrieval

Why BM25?

Why dense retrieval?

Why hybrid?

Embedding

Why did you choose this embedding model?

RAG

Why does RAG improve your baseline?

Chunking

Why 512 instead of 256?

Reranking

Why does reranking help?

Evaluation

How do you know your system is better?

Engineering

How would you scale it?

DevOps

How do you deploy it?

Failure

What doesn't work?

Future

What would you improve?

Nếu bạn trả lời được những câu này bằng **kết quả experiment của chính bạn**, project sẽ rất có giá trị.

Toàn bộ project thực chất là chuỗi này

Bạn có thể lưu nguyên cái này làm **master checklist**:

1. Problem Definition
   ↓
2. Domain Research
   ↓
3. Literature / Technology Research
   ↓
4. Dataset Collection
   ↓
5. Data Preprocessing
   ↓
6. Evaluation Dataset
   ↓
7. Simple Baseline
   ↓
8. BM25 / Traditional Retrieval
   ↓
9. Dense Retrieval
   ↓
10. Chunking Experiments
   ↓
11. Embedding Experiments
   ↓
12. Vector Index
   ↓
13. LLM Baseline
   ↓
14. RAG
   ↓
15. Hybrid Retrieval
   ↓
16. Reranking
   ↓
17. Query Understanding
   ↓
18. Error Analysis
   ↓
19. Improve System
   ↓
20. Structured Output
   ↓
21. Backend API
   ↓
22. Web Application

↓
23. Testing
↓
24. Experiment Tracking
↓
25. Dataset Versioning
↓
26. Docker
↓
27. CI
↓
28. CD
↓
29. Deployment
↓
30. Monitoring
↓
31. AI Observability
↓
32. Security
↓
33. Final Evaluation
↓
34. Error Analysis
↓
35. Documentation
↓
36. Technical Report
↓
37. Demo
↓
38. Interview Preparation